STM32_1

실험 전 보고서

STM32_1 · 인터럽트와 PWM

실험 주제 : STM32의 인터럽트와 PWM 기능을 활용하여 버튼 입력을 효율적으로 처리하고 다양한 듀티비의 신호를 생성하는 방법 학습

실험 대상 : STM32 NUCLEO-G474RE의 버튼 인터럽트, TIM6 타이머 인터럽트, PWM 출력(PB4), LED 제어 기능

작성일 : 2026년 10월 18일

검증 환경 : VS Code STM32

성명 : 권진혁

학과 : 전자전기컴퓨터공학부

학번 : 2023440007

1. 실험목적

본 실험의 목적은 STM32 마이크로컨트롤러의 인터럽트(Interrupt)와 PWM(Pulse Width Modulation) 기능을 이해하고 활용하는 데 있다. 외부 인터럽트를 이용하여 버튼 입력을 효율적으로 처리하는 방법을 학습하고, 타이머 기반 PWM 신호를 생성하여 LED의 밝기나 출력 신호를 제어하는 과정을 실습한다. 또한 폴링 방식과 인터럽트 방식의 차이를 비교하고, STM32의 하드웨어 자원을 활용한 실시간 제어 기법을 익힘으로써 임베디드 시스템 설계 능력을 향상시키고자 한다.

2. 관련이론

· 인터럽트(Interrupt)

인터럽트는 마이크로컨트롤러가 특정 이벤트를 감지했을 때 현재 수행 중인 작업을 잠시 중단하고, 미리 지정된 인터럽트 서비스 루틴(ISR, Interrupt Service Routine)을 실행한 후 원래 작업으로 복귀하는 기능이다.
일반적인 폴링(Polling) 방식은 CPU가 지속적으로 입력 신호를 확인해야 하지만, 인터럽트 방식은 이벤트가 발생할 때만 CPU가 동작하므로 시스템 자원을 효율적으로 사용할 수 있다.

- 인터럽트 동작 과정

1) 프로그램 실행
2) 인터럽트 발생
3) 현재 작업 상태 저장
4) ISR 실행
5) 원래 프로그램으로 복귀

· 외부 인터럽트(External Interrupt)

외부 인터럽트는 버튼, 스위치 등의 외부 입력 신호에 의해 발생하는 인터럽트이다. STM32에서는 EXTI를 사용하여 GPIO 핀의 상태 변화(Rising Edge, Falling Edge)를 감지한다.

예를 들어 사용자가 버튼을 누르면 인터럽트가 발생하고, CPU는 즉시 ISR을 실행하여 LED 상태 변경 등의 작업을 수행할 수 있다.

· 타이머(Timer)

타이머는 내부 클럭을 이용하여 시간을 측정하거나 주기적인 이벤트를 발생시키는 하드웨어 모듈이다. STM32의 타이머는 다음과 같은 기능을 수행할 수 있다.

- 주기적 인터럽트 생성
- PWM 신호 생성
- 시간 측정
- 카운터 기능

타이머의 카운터 값은 클럭에 따라 증가하며, 설정된 ARR(Auto Reload Register) 값에 도달하면 다시 0부터 카운트한다.

· PWM(Pulse Width Modulation)

PWM은 일정한 주기를 가지는 디지털 신호의 ON 시간과 OFF 시간의 비율을 조절하여 평균 출력 전압을 제어하는 기술이다. PWM 신호의 주요 파라미터는 주파수(Frequency)와 듀티비(Duty Cycle)이다.

- 주기(Period)

PWM 한 사이클이 완료되는 시간을 의미한다. $T = \frac{1}{f}$ (T : 주기, f : 주파수)

- 듀티비(Duty Cycle)

전체 주기 중 출력이 HIGH 상태인 시간의 비율을 의미한다. $Duty(\%) = \frac{T_{ON}}{T} \times 100$

· PWM을 이용한 LED 제어

LED는 매우 빠른 속도로 ON/OFF 되어도 사람이 이를 평균 밝기로 인식한다. 예를 들어

- 듀티비 20% → 어두움
- 듀티비 50% → 중간 밝기
- 듀티비 80% → 밝음

따라서 PWM의 듀티비를 조절하면 아날로그 회로 없이도 LED 밝기를 제어할 수 있다.

· STM32에서 PWM 생성 원리

STM32는 타이머의 카운터와 CCR(Capture Compare Register)을 이용하여 PWM 신호를 생성한다.

· 인터럽트와 PWM의 활용

인터럽트와 PWM을 함께 사용하면 사용자 입력에 따라 PWM 출력이 실시간으로 변경되는 시스템을 구현할 수 있다. 예를 들어 버튼 인터럽트가 발생할 때마다

- 20%
- 40%
- 60%
- 80%
- 100%

순으로 PWM 듀티비를 변경하면 LED 밝기가 단계적으로 변하는 것을 확인할 수 있다.

3. 실험 장비

1)STM32
2) PC
3) STM32CubeIDE
4) USB 케이블
5) 브레드보드
6) LED
7) 푸시 버튼
8) 점퍼선
9) 오실로스코프

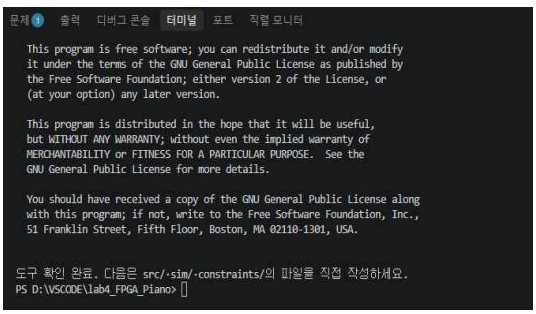
4. FPGA 실습 - Piano

· 실험 목적

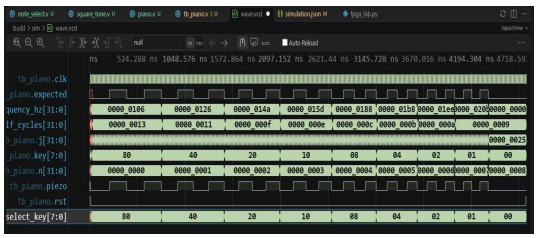
FPGA를 이용하여 피아노 기능을 구현함으로써 디지털 회로 설계 능력을 향상시키고, Verilog HDL을 활용한 조합논리회로 및 순차논리회로의 동작 원리를 이해하는 데 있다. 또한, 스위치 입력에 따라 각 음계에 해당하는 주파수를 생성하고 피에조 스피커로 출력하는 과정을 통해 카운터, 클록 분주, 모듈 설계 및 계층적 설계 기법을 학습한다.

· 핵심 코드 : Git-Hub 참고

· 실행 결과







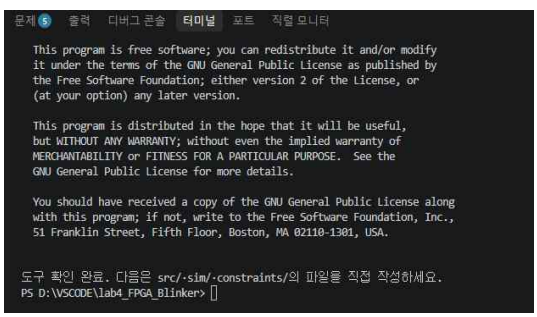
5. FPGA 실습 - 신호등 제어

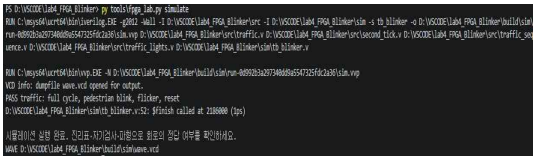
· **실험 목적**

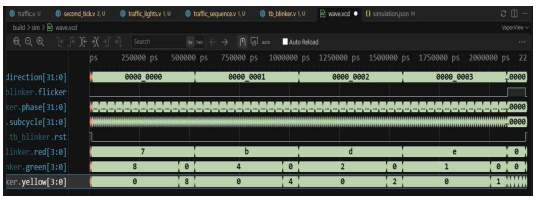
FPGA를 이용하여 신호등 제어 시스템을 구현함으로써 순차논리회로의 동작 원리를 이해하고, Verilog HDL을 활용한 상태기계(FSM, Finite State Machine) 설계 방법을 학습하는 데 있다. 또한, 클록 신호를 이용한 시간 제어와 상태 전이를 구현하고, 이를 통해 실제 교통 신호 제어 시스템의 동작 원리를 이해하며 디지털 시스템 설계 능력을 향상시키고자 한다.

· **핵심 코드** : Git-Hub 참고

· **실행 결과**







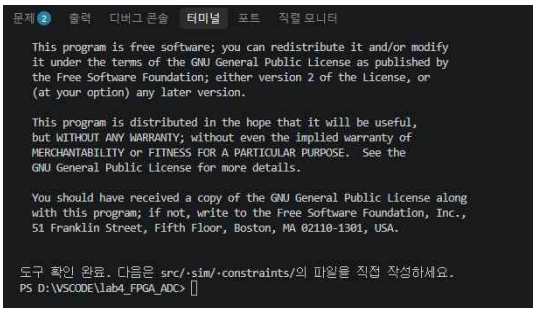
6. FPGA 실습 - ADC

· **실험 목적**

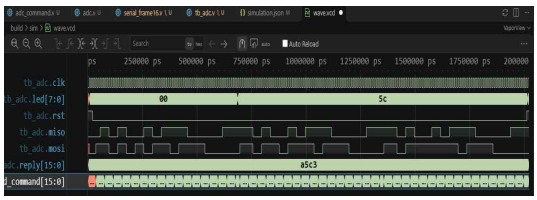
FPGA를 이용하여 ADC(Analog-to-Digital Converter)의 직렬 통신 동작 원리를 이해하고, Verilog HDL을 활용하여 채널 선택 명령 전송과 변환 데이터 수신 과정을 구현하는 데 있다. 또한, ADC로부터 수신한 12비트 변환 데이터를 저장하고 LED를 통해 출력함으로써 직렬 데이터 통신, 데이터 처리 및 모듈 간 연동 방법을 학습하고 디지털 시스템 설계 능력을 향상하고자 한다.

· **핵심 코드** : Git-Hub 참고

· **실행 결과**







7. STM32_1 : 과제3

· 핵심 코드 및 설명

```c
#include "ece2_474.h"

volatile uint32_t led_interval = 10U;
```

led_interval 변수는 LED의 점멸 주기를 저장하기 위한 전역 변수이다. TIM6 인터럽트가 100ms마다 발생하므로, 해당 변수에는 LED 상태를 변경하기까지 필요한 인터럽트 횟수를 저장한다. 초기값인 10은 약 1초를 의미하며, 버튼 입력에 따라 20(2초), 40(4초), 80(8초)으로 변경된다. 전역 변수로 선언하여 버튼 인터럽트 함수와 타이머 인터럽트 함수가 동일한 값을 공유할 수 있도록 하였다.

```c
HAL_TIM_Base_Start_IT(htim: &htim6);

    /* TIM3: PB4에서 1 kHz PWM을 출력합니다. 비교값 500은 50%입니다. */
HAL_TIM_PWM_Start(htim: &htim3, Channel: TIM_CHANNEL_1);
HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 500U);
```

HAL_TIM_Base_Start_IT() 함수는 TIM6 타이머를 인터럽트 모드로 동작시키기 위해 사용하였다. TIM6은 100ms마다 인터럽트를 발생시키며, 인터럽트 발생 시 tim6_isr_p4() 함수가 호출된다. 이를 통해 메인 루프에서 지속적으로 시간을 확인하는 폴링 방식 대신 인터럽트 기반의 주기 제어를 구현하여 시스템 자원을 효율적으로 사용할 수 있다.

```c
static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();

    if(now_ms - last_ms >= 200U)
    {
        last_ms = now_ms;

        switch(led_interval)
        {
        case 10U:
            led_interval = 20U;
            break;

        case 20U:
            led_interval = 40U;
            break;

        case 40U:
            led_interval = 80U;
            break;

        default:
            led_interval = 10U;
            break;
        }
    }
}
```

button_isr_p7() 함수는 사용자 버튼이 눌렸을 때 실행되는 인터럽트 서비스 루틴이다. 함수 내부에서는 HAL_GetTick()을 이용하여 버튼 입력 간의 시간을 측정하고, 200ms 이내에 발생하는 중복 입력을 무시하는 디바운싱 기능을 수행한다. 또한 버튼이 정상적으로 입력되면 led_interval 값을 10, 20, 40, 80 순으로 변경하여 LED 점멸 주기를 단계적으로 증가시키고, 마지막 단계에서는 다시 초기값으로 복귀하도록 구현하였다.

```c
static void tim6_isr_p4(void)
{
    static uint32_t count = 0;

    if(++count >= led_interval)
    {
        count = 0;

        HAL_GPIO_TogglePin(
            GPIOx: LED_ece2_GPIO_Port,
            GPIO_Pin: LED_ece2_Pin
        );
    }
}
```

tim6_isr_p4() 함수는 TIM6 인터럽트가 발생할 때마다 호출되는 함수이다. 내부 카운터 변수를 이용하여 인터럽트 발생 횟수를 누적하고, 누적 횟수가 led_interval 값에

도달하면 LED의 상태를 반전시킨다. 이를 통해 버튼 인터럽트에서 설정된 주기에 맞추어 LED가 1초, 2초, 4초, 8초 간격으로 점멸하도록 구현하였다. 또한 인터럽트 내부에서는 짧은 작업만 수행하여 실시간 제어 시스템의 효율성을 유지하도록 설계하였다.

· 예상 결과

TIM6 타이머 인터럽트가 100ms 주기로 발생하며 설정된 횟수만큼 인터럽트가 누적될 때마다 LED의 상태가 토글되는 것을 확인할 수 있다. 초기 상태에서는 LED가 약 1초 간격으로 점멸하고, 사용자 버튼을 누를 때마다 점멸 주기가 2초, 4초, 8초 순으로 증가한 후 다시 1초로 초기화된다. 또한 버튼 입력에 디바운싱 처리가 적용되어 접점 잡음에 의한 오동작이 발생하지 않으며, 폴링 방식이 아닌 인터럽트 방식을 사용함으로써 메인 루프의 부하를 줄이면서도 정확한 시간 제어와 안정적인 LED 동작을 확인할 수 있을 것으로 예상된다.

7. STM32_1 : 과제4

· 핵심 코드 및 해설

```c
#include "ece2_474.h"

#include "main.h"

volatile uint32_t pwm_compare = 250U;
```

pwm_compare 변수는 PWM 신호의 듀티비를 결정하는 비교값(CCR 값)을 저장하기 위한 전역 변수이다. TIM3의 주기를 1000으로 설정한 상태에서 250은 25%, 500은 50%, 750은 75%의 듀티비를 의미한다. 버튼 인터럽트와 PWM 출력 설정 함수가 동일한 값을 사용해야 하므로 전역 변수로 선언하였으며, 버튼 입력에 따라 PWM 출력 비율이 변경될 수 있도록 하였다.

```c
void ece2_474_init(void)
{
    /* 필요한 기능만 주석을 해제합니다. 기본 핀·통신 설정은 이미 끝난 상태입니다.
     * VS Code에서 해당 줄에 커서를 놓거나 여러 줄을 선택하고 Ctrl + /를 누르면
     * 줄 주석(//)을 해제하거나 다시 설정할 수 있습니다.
     */

    /* TIM6: 100 ms마다 tim6_isr_p4()를 호출합니다. */
    // HAL_TIM_Base_Start_IT(&htim6);

    /* TIM3: PB4에서 1 kHz PWM을 출력합니다. 비교값 500은 50%입니다. */
    HAL_TIM_PWM_Start(htim: &htim3, Channel: TIM_CHANNEL_1);

    __HAL_TIM_SET_COMPARE(
        &htim3,
        TIM_CHANNEL_1,
        pwm_compare
    );
```

HAL_TIM_PWM_Start() 함수는 TIM3의 PWM 출력을 시작하기 위해 사용하였다. 또한 __HAL_TIM_SET_COMPARE() 함수는 타이머의 비교 레지스터(CCR) 값을 설정하여 듀티비를 결정한다. 초기값으로 pwm_compare를 적용하여 실험 시작 시 25% 듀티비의 PWM 신호가 출력되도록 하였으며, 이후 버튼 입력에 따라 변경되는 값을 즉시 PWM 출력에 반영할 수 있도록 구성하였다.

```c
static void button_isr_p7(void)
{
    static uint32_t last_ms = 0;
    uint32_t now_ms = HAL_GetTick();

    if(now_ms - last_ms >= 200U)
    {
        last_ms = now_ms;

        switch(pwm_compare)
        {
        case 250U:
            pwm_compare = 500U;
            break;

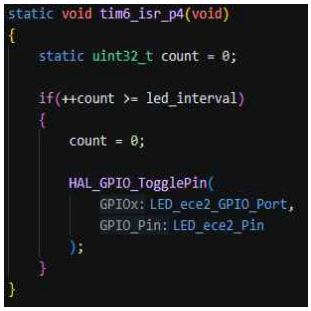
        case 500U:
            pwm_compare = 750U;
            break;

        default:
            pwm_compare = 250U;
            break;
        }

        __HAL_TIM_SET_COMPARE(
            &htim3,
            TIM_CHANNEL_1,
            pwm_compare
        );
    }
}
```

button_isr_p7() 함수는 사용자 버튼이 눌렸을 때 실행되는 인터럽트 서비스 루틴이다. 함수 내부에서는 HAL_GetTick()을 이용한 디바운싱 처리를 수행하여 스위치 접점 잡음으로 인한 오동작을 방지하였다. 버튼이 정상적으로 입력되면 pwm_compare 값을 250, 500, 750 순으로 변경하고, 마지막 단

계 이후에는 다시 250으로 초기화한다. 또한 변경된 값을 __HAL_TIM_SET_COMPARE() 함수에 적용하여 PWM 듀티비가 즉시 갱신되도록 구현하였다.



```
static void tim6_isr_p4(void)
{
    static uint32_t count = 0;

    if(++count >= led_interval)
    {
        count = 0;

        HAL_GPIO_TogglePin(
            GPIOx: LED_ece2_GPIO_Port,
            GPIO_Pin: LED_ece2_Pin
        );
    }
}
```

tim6_isr_p4() 함수는 TIM6 인터럽트가 발생할 때마다 호출되는 함수이다. 내부 카운터 변수를 이용하여 인터럽트 발생 횟수를 누적하고, 누적 횟수가 led_interval 값에 도달하면 LED의 상태를 반전시킨다. 이를 통해 버튼 인터럽트에서 설정된 주기에 맞추어 LED가 1초, 2초, 4초, 8초 간격으로 점멸하도록 구현하였다. 또한 인터럽트 내부에서는 짧은 작업만 수행하여 실시간 제어 시스템의 효율성을 유지하도록 설계하였다.

· 예상 결과

TIM3가 PWM 신호를 출력하는 상태에서 사용자 버튼을 누를 때마다 비교값(CCR)이 변경되어 PWM 듀티비가 25%(250) → 50%(500) → 75%(750) → 25%(250) 순으로 반복적으로 변화하는 것을 확인할 수 있다. 이에 따라 PWM 출력 핀(PB4)에 연결된 LED는 단계적으로 밝기가 증가하다가 초기 상태로 돌아가며, 오실로스코프로 측정할 경우 PWM 주기는 일정하게 유지되고 HIGH 구간의 비율만 변경되는 파형을 관찰할 수 있다. 또한, 디바운싱 처리가 적용되어 버튼을 한 번 눌렀을 때 듀티비가 한 단계씩만 변경되며, 인터럽트 방식을 사용함으로써 효율적이고 안정적인 PWM 제어가 가능함을 확인할 수 있을 것으로 예상된다.

8. Git-Hub

https://github.com/UnnamedDays/ECE-II-Project

056a192

8. 출처

1) STMicroelectronics, STM32G4 Series Reference Manual (RM0440), STMicroelectronics.

2) 전자전기컴퓨터공학부, STM32_1 인터럽트와 PWM 실험 교안, 서울시립대학교.

3) STMicroelectronics, STM32Cube HAL and Low-Layer Drivers User Manual, STMicroelectronics.